arXiv:1811.05701v3 [cs.CL] 19 Feb 2019

Plan-And-Write: Towards Better Automatic Storytelling

Lili Yao[1,3*], **Nanyun Peng**[2*], **Ralph Weischedel**[2], **Kevin Knight**[2], **Dongyan Zhao**[1] and **Rui Yan**[1†]
liliyao@tencent.com, {npeng,weisched,knight}@isi.edu
{zhaodongyan,ruiyan}@pku.edu.cn
[1] Institute of Computer Science and Technology, Peking University
[2] Information Sciences Institute, University of Southern California, [3] Tencent AI Lab

Abstract

Automatic storytelling is challenging since it requires generating long, coherent natural language to describes a sensible sequence of events. Despite considerable efforts on automatic story generation in the past, prior work either is restricted in plot planning, or can only generate stories in a narrow domain. In this paper, we explore open-domain story generation that writes stories given a title (topic) as input. We propose a *plan-and-write* hierarchical generation framework that first plans a storyline, and then generates a story based on the storyline. We compare two planning strategies. The *dynamic* schema interweaves story planning and its surface realization in text, while the *static* schema plans out the entire storyline before generating stories. Experiments show that with explicit storyline planning, the generated stories are more diverse, coherent, and on topic than those generated without creating a full plan, according to both automatic and human evaluations.

Introduction

A narrative or story is anything which is told in the form of a causally/logically linked set of events involving some shared characters (Mostafazadeh et al. 2016a). Automatic storytelling requires composing coherent natural language texts that describe a sensible sequence of events. This seems much harder than text generation where a plan or knowledge fragment already exists. Thus, story generation seems an ideal testbed for advances in general AI. Prior research on story generation mostly focused on automatically composing a sequence of events that can be told as a story by plot planning (Lebowitz 1987; Perez and Sharples 2001; Porteous and Cavazza 2009; Riedl and Young 2010; Li et al. 2013) or case-based reasoning (Turner 1994; Gervas et al. 2005). These approaches rely heavily on human annotation and/or are restricted to limited domains. Moreover, most prior work is restricted to the abstract story representation level without surface realization in natural language.

In this paper, we study generating natural language stories from any given title (topic). Inspired by prior work on dialog planning (Nayak et al. 2017) and narrative planning (Riedl and Young 2010), we propose to decompose story generation into two steps: 1) story planning which generates plots, and 2) surface realization which composes natural language text based on the plots. We propose a *plan-and-write* hierarchical generation framework that combines plot planning and surface realization to generate stories from titles.

Title (Given)	The Bike Accident
Storyline (Extracted)	Carrie → bike → sneak → nervous → leg
Story (Human Written)	Carrie had just learned how to ride a bike. She didn't have a bike of her own. Carrie would sneak rides on her sister's bike. She got nervous on a hill and crashed into a wall. The bike frame bent and Carrie got a deep gash on her leg.

Table 1: An example of title, storyline and story in our system. A storyline is represented by an ordered list of words.

One major challenge for our framework is how to represent and obtain annotations for story plots so that a reasonable generative model can be trained to plan story plots. Li et al. [2013] introduces plot graphs which contain events and their relations to represent a storyline. Plot graphs are comprehensive representations of story plots, however, the definition and curation of such plot graphs require highly specialized knowledge and significant human effort. On the other hand, in poetry composition, Wang et al. [2016] provides a sequence of words to guide poetry generation. In conversational systems, Mou et al. [2016] takes keywords as the main gist of the reply to guide response generation. We take a similar approach to represent a story plot with a sequence of words. Specifically, we use the order that the words appear in the story to approximate a storyline. Table 1 shows an example of the title, storyline, and story.

Though this representation seems to over-simplify story plots, it has several advantages. First, because the storyline representation is simple, there are many reliable tools to extract high-quality storylines from existing stories and thus automatically generate training data for the plot planning model. Our experiments show that by training plot planning models on automatically extracted storylines, we

*Equal contribution: Lili Yao and Nanyun Peng
†Corresponding author: Rui Yan (ruiyan@pku.edu.cn)
Copyright © 2019, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

can generate better stories without additional human annotation. Moreover, with this simple and interpretable storyline representation, it is possible to compare the efficiency of different plan-and-write strategies. Specifically, we explore two paradigms that seem to mimic human practice in real world story writing[1] (Alarcon 2010). The *dynamic* schema adjusts the plot improvisationally while writing progresses. The *static* schema plans the entire plot before writing. We summarize the contributions of the paper as follows:

- We propose a *plan-and-write* framework that leverages storylines to improve the diversity and coherence of the generated story. Two strategies: *dynamic* and *static* planning are explored and compared under this framework.
- We develop evaluation metrics to measure the diversity of the generated stories, and conduct novel analysis to examine the importance of different aspects of stories for human evaluation.
- Experiments show that the proposed plan-and-write model generates more diverse, coherent, and on-topic stories than those without planning [2].

Plan-and-Write Storytelling

In this paper, we propose a plan-and-write framework to generate stories from given titles. We posit that storytelling systems can benefit from storyline planning to generate more coherent and on-topic stories. An additional benefit of the plan-and-write schema is that human and computer can interact and collaborate on the (abstract) storyline level, which can enable many potentially enjoyable interactions. We formally define the input, output, and storyline of our approach as follows.

Problem Formulation

Input: A title $\mathbf{t} = \{t_1, t_2, ..., t_n\}$ is given to the system to constrain writing, where t_i is the i-th word in the title.

Output: The system generates a story $\mathbf{s} = \{s_1, s_2, ..., s_m\}$ based on a title, where s_i denotes a sentence in the story.

Storyline: The system plans a storyline $\mathbf{l} = \{l_1, l_2, ..., l_m\}$ as an intermediate step to represent the plot of a story. We use a sequence of words to represent a storyline, therefore, l_i denotes a word in a storyline.

Given a title, the plan-and-write framework always plans a storyline. We explore two variations of this framework: the *dynamic* and the static schema.

Storyline Preparation

To obtain training data for the storyline planner, we extract sequences of words from existing story corpora to compose storylines. Specifically, we extract one word from each sentence of a story to form a storyline[3]. We adopt the RAKE algorithm (Rose et al. 2010), which combines several word frequency based and graph-based metrics to weight the importance of the words. We extract the most important word from each sentence as a story's storyline.

Methods

We adopt neural generation models to implement our plan-and-write framework, as they have been shown effective in many text generation tasks such as machine translation (Bahdanau, Cho, and Bengio 2015), and dialogue systems (Shang, Lu, and Li 2015). Figure 1 demonstrates the workflow of our framework. We now describe the two plan-and-write strategies we explored.

Dynamic Schema

The dynamic schema emphasizes flexibility. As shown in Figure 2a, it generates the next word in the storyline and the next sentence in the story at each step. In both cases, the existing storyline and previously generated sentences are given to the model to move one step forward.

Storyline Planning The storyline is planned out based on the context (the title and previously generated sentences are taken as context) and the previous word in the storyline. We formulate it as a content-introducing generation problem, where the new content (the next word in the storyline) is generated based on the context and some additional information (the most recent word in the storyline). Formally, let $\mathbf{ctx} = [\mathbf{t}, \mathbf{s}_{1:i-1}]$ denotes the context, where $s_{1:i-1}$ denotes for the first i-1 sentences in the story. We model $p(l_i|\mathbf{ctx}, l_{i-1}; \theta)$. We implement the content-introducing method proposed by Yao et al. [2017], which first encodes context into a vector using a bidirectional gated recurrent unit (BiGRU), and then incorporates the auxiliary information, in this case the previous word in the storyline, into the decoding process. Formally, hidden vectors for context are computed as $\widetilde{\mathbf{h}}_{ctx} = Encode_{ctx}(\mathbf{ctx}) = [\overrightarrow{\mathbf{h}_{ctx}}; \overleftarrow{\mathbf{h}_{ctx}}]$, where $\overrightarrow{\mathbf{h}_{ctx}}$ and $\overleftarrow{\mathbf{h}_{ctx}}$ are the hidden vectors produced by a forward and a backward GRU, respectively. $[;]$ denotes element-wise concatenation. The conditional probability is computed as:

$$h_y = \text{GRU}(\text{BOS}, C_{att}), h_w = \text{GRU}(l_{i-1}, C_{att})$$
$$h_y^{'} = \tanh(W_1 h_y), h_w^{'} = \tanh(W_2 h_w)$$
$$k = \sigma(W_k[h_y^{'}; h_w^{'}])$$
$$p(l_i|\mathbf{ctx}, l_{i-1}) = g(k \circ h_y + (1-k) \circ h_w)$$

BOS denotes the beginning of decoding. C_{att} represents the attention-based context computed from $\widetilde{\mathbf{h}}_{ctx}$. g(·) denotes a multilayer perceptron (MLP).

Story Generation The story is generated incrementally by planning and writing alternately. We formulate it as another content-introducing generation problem which generates a

[1] Some discussions on Quora: https://www.quora.com/How-many-times-does-a-writer-edit-his-first-draft

[2] Code and appendix will be available at https://bitbucket.org/VioletPeng/language-model

[3] For this pilot study, we assume each word l_i in a storyline corresponds to a sentence s_i in a story.

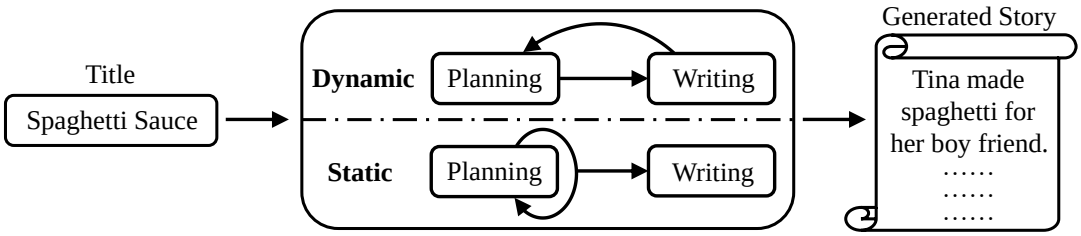
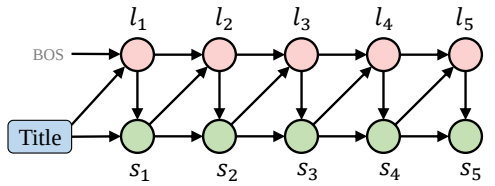
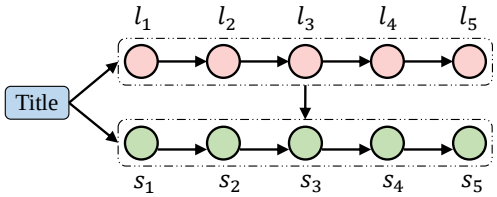


Figure 1: An overview of our system.



(a) Dynamic schema work-flow.



(b) Static schema work-flow.

Figure 2: An illustration of the dynamic and static plan-and-write work-flow. l_i denotes a *word* in a storyline and s_i denotes a *sentence* in a story.

story sentence based on both the context and an additional storyline word as a cue. The model structure is exactly the same as for storyline generation. However, there are two differences between storyline and story generation. On one hand, the former aims to generate a word while the latter generates a variable-length sequence. On the other hand, the auxiliary information they use is different.

Formally, the model is trained to minimize the negative log-probability of training data:

$$\mathcal{L}(\theta)_{dyna} = \frac{1}{N}\sum_{j=1}^{N}\left[-\log\prod_{i=1}^{m} p(s_i|\mathbf{ctx}, l_i)\right]_j \quad (1)$$

where N is the number of stories in training data; m denotes the number of sentences in a story. Given the extracted storylines as described in the previous Section, the storyline and story generation models are trained separately. End-to-end generation is conducted in a pipeline fashion.

Static Schema

The static schema is inspired by sketches that writers usually draw before they flesh out the whole story. As illustrated in Figure 2b, it first generates a whole storyline which does not change during story writing. This sacrifices some flexibility in writing, but could potentially enhance story coherence as it provides "look ahead" for what happens next.

Storyline Planning Differing from the dynamic schema, storyline planning for static schema is solely based on the title $\mathbf{t}$. We formulate it as a conditional generation problem, where the probability of generating each word in a storyline depends on the previous words in the storyline and the title. Formally, we model $p(l_i|\mathbf{t}, l_{1:i-1}; \theta)$. We adopt a sequence-to-sequence (Seq2Seq), conditional generation model that first encodes the title into a vector using a bidirectional long short-term memory network (BiLSTM), and generates words in the storyline using another single-directional LSTM. Formally, the hidden vector $\tilde{\mathbf{h}}$ for a title is computed as $\tilde{\mathbf{h}} = Encode(\mathbf{t}) = [\overrightarrow{\mathbf{h}}; \overleftarrow{\mathbf{h}}]$, and the conditional probability is given by:

$$p(l_i|\mathbf{t}, l_{1:i-1}; \theta) = g(\text{LSTM}_{\text{att}}(\tilde{\mathbf{h}}, l_{i-1}, \mathbf{h}^{\text{dec}}_{i-1}))$$

where LSTM_{att} denotes a cell of the LSTM with attention mechanism (Bahdanau, Cho, and Bengio 2015); $\mathbf{h}^{\text{dec}}_{i-1}$ stands for the decoding hidden state. g(·) again denotes a MLP.

Story Generation The story is generated after the full storyline is planned. We formulate it as another conditional generation problem. Specifically, we train a Seq2Seq model that encodes both the title and the planned storyline into a low-dimensional vector by first concatenating them with a special symbol <EOT> in between, and encode them with BiLSTMs: $\tilde{\mathbf{h}}_{tl} = Encode_{tl}([\mathbf{t}, \mathbf{l}]) = [\overrightarrow{\mathbf{h}_{tl}}; \overleftarrow{\mathbf{h}_{tl}}]$. The Seq2Seq model is then trained to minimize the negative log-probability of the stories in the training data:

$$\mathcal{L}(\theta)_{static} = \frac{1}{N}\sum_{j=1}^{N}\left[-\log\prod_{i=1}^{m} p(s_i|\tilde{\mathbf{h}}_{tl}, s_{1:i-1})\right]_j \quad (2)$$

Storyline Optimization

One common problem for neural generation models is the repetition in generated results (Li et al. 2016). We observe repetition initially in both the generated storyline (repeated words) and story (repeated phrases and sentences). An advantage of the storyline layer is that given the compact and interpretable representation of the storyline, we can easily apply heuristics to reduce repetition[4]. Specifically, we forbid any word to appear twice when generating a storyline.

Experimental Setup

Dataset

We conduct the experiments on the ROCStories corpus (Mostafazadeh et al. 2016a). It contains 98,162 short

[4]It is important to avoid repetition in the generated stories too. However, it is hard to automatically detect repetition in stories. Optimizing storylines can indirectly reduce repetition in stories.